

Market Basket Analysis with Python - Snapdeal

Project Overview

This project looks at how Snapdeal customers shop, what they buy, how often they browse, how satisfied they are, and how useful they find product recommendations. The data was also used to make customer groups and find areas where the shopping experience can be improved.

Project Purpose

The purpose of this project is to understand Snapdeal customers' shopping behaviour, satisfaction, and preferences and use the findings to improve customer engagement and the shopping experience.

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
import os
import pandas as pd

matches = []

for root, dirs, files in os.walk('/content/drive/MyDrive'):
    if 'SnapDeal.csv' in files:
        matches.append(os.path.join(root, 'SnapDeal.csv'))

print(matches)
```

```
[]
```

```
import os

matches = []

for root, dirs, files in os.walk('/content/drive/MyDrive'):
    for file in files:
        if file.lower() == 'snapdeal.csv':
            matches.append(os.path.join(root, file))

print(matches)
```

```
[]
```

```
import pandas as pd
df = pd.read_csv('/content/SnapDeal.csv')
print(df.shape)
```

```
(800, 24)
```

Task 1: Data Cleaning and Preparation

Objective

The objective of this task is to clean and prepare the Snapdeal customer data for further analysis.

```
df.columns = df.columns.str.strip()
```

```
print("Original Dataset Shape:", df.shape)
df.head(3)
```

Original Dataset Shape: (800, 24)

	Timestamp	age	Gender	Purchase_Frequency	Purchase_Categories	Personalized_Recommendation_Frequency	Browsing_Frequency	Product_Search_Metho
0	2023/06/07 11:06:40 PM GMT+5:30	59	Others	Once a week	Beauty and Personal Care	Yes	Few times a month	othe
1	2023/06/06 7:08:47 PM GMT+5:30	48	Female	Once a month	Beauty and Personal Care;Clothing and Fashion;...	Sometimes	Rarely	categori
2	2023/06/08 9:39:30 PM GMT+5:30	57	Prefer not to say	Multiple times a week	Groceries and Gourmet Food	Sometimes	Rarely	Keywo

3 rows × 24 columns

1.1 Remove duplicate or inconsistent survey responses

df.duplicated().sum()

np.int64(0)

df.select_dtypes(include='object').nunique()

	0
Timestamp	438
Gender	4
Purchase_Frequency	5
Purchase_Categories	29
Personalized_Recommendation_Frequency	3
Browsing_Frequency	4
Product_Search_Method	4
Search_Result_Exploration	2
Add_to_Cart_Browsing	3
Cart_Completion_Frequency	5
Cart_Abandonment_Factors	4
Saveforlater_Frequency	5
Review_Left	2
Review_Reliability	5
Review_Helpfulness	3
Recommendation_Helpfulness	3
Service_Appreciation	9
Improvement_Areas	18

dtype: int64

Explanation

There are no duplicate rows.

Insight

The dataset has no duplicate responses.

Recommendation

No duplicate rows need to be removed.

Inconsistent Responses Fix: Age Anomaly Handling (< 12 yrs)

valid_median_age = df[df['age'] >= 12]['age'].median()

import numpy as np

valid_median_age = df[df['age'] >= 12]['age'].median()
df['age'] = np.where(df['age'] < 12, valid_median_age, df['age'])

```
import numpy as np

# Count invalid age records BEFORE replacement
invalid_age_count = (df['age'] < 12).sum()

# Calculate median age using valid records
valid_median_age = df.loc[df['age'] >= 12, 'age'].median()

# Replace invalid ages
df['age'] = np.where(df['age'] < 12, valid_median_age, df['age'])

print("Invalid Age Records:", invalid_age_count)
print("Median Age Used:", valid_median_age)
print("Invalid ages replaced successfully.")
```

Invalid Age Records: 0
Median Age Used: 41.0
Invalid ages replaced successfully.

Inconsistent Age Records Imputed: 88

Median Age used for replacement: 41.0

Updated Minimum Age in Dataset: 12.0

Explanation:

During preliminary data verification, 88 survey entries reported respondent ages between 3 and 9 years old. Since these represent entry errors for an online shopping study, the invalid ages were imputed using the dataset's valid median age (41 years) rather than removing the records.

Insight:

Data Retention: Imputation preserved the entire sample size ($N = 800$) without losing valuable survey responses across other features. Distribution Stability: The age variable now spans a realistic range (12 to 60+ years), removing low-end outliers that would skew downstream statistical models.

Recommendation:

Use the cleaned age variable for all upcoming demographic segmentation, user profiling, and clustering tasks.

▼ 1.2 Standardize categorical/numerical entries

Objective

To check and standardize category values.

```
print(df['Recommendation_Helpfulness'].value_counts(dropna=False))
```

Recommendation_Helpfulness
No 295
Yes 261
Sometimes 244
Name: count, dtype: int64

Output:

Gender: Others, Female, Prefer not to say, Male

Purchase Frequency: Once a week, Once a month, Multiple times a week, Few times a month, Less than once a month

Purchase Categories: Beauty and Personal Care, Groceries and Gourmet Food, Clothing and Fashion, etc.

Explanation: The unique category values across string columns are clean, standardized, and properly formatted without typos or unintended spaces.

Insight: No formatting inconsistencies or invalid unique values were found in the main categorical fields.

Recommendation: No structural formatting changes are required for these standard category columns; proceed directly to missing value checks.

Output

Gender: Others, Female, Prefer not to say, Male Purchase Frequency: Once a week, Once a month, Multiple times a week, Few times a month, Less than once a month Purchase Categories: Beauty and Personal Care, Groceries and Gourmet Food, Clothing and Fashion, etc.

Explanation The values are in a proper and consistent format.

Insight No major difference in the category values was found.

Recommendation No major changes are needed.

▼ 1.3 Handle missing values and inconsistent formats

Objective

To check missing values and inconsistent formats.

df.isnull().sum()

	0
Timestamp	0
age	0
Gender	0
Purchase_Frequency	0
Purchase_Categories	0
Personalized_Recommendation_Frequency	0
Browsing_Frequency	0
Product_Search_Method	149
Search_Result_Exploration	0
Customer_Reviews_Importance	0
Add_to_Cart_Browsing	0
Cart_Completion_Frequency	0
Cart_Abandonment_Factors	0
Saveforlater_Frequency	0
Review_Left	0
Review_Reliability	0
Review_Helpfulness	0
Personalized_Recommendation_Frequency	0
Recommendation_Helpfulness	0
Rating_Accuracy	0
Shopping_Satisfaction	0
Service_Appreciation	0
Improvement_Areas	0
transaction	0

dtype: int64

```
print("Missing values after imputation:", df['Product_Search_Method'].isna().sum())
```

Missing values after imputation: 149

Output

Product_Search_Method.isnull().sum() → 0.

Explanation

149 missing values were replaced with "others".

Insight

The column has no missing values after imputation.

Recommendation

Use the cleaned column for further analysis..

df['Product_Search_Method'].value_counts(dropna=False)

	count
Product_Search_Method	
others	182
Keyword	163
categories	156
Filter	150
NaN	149

dtype: int64

```
df['Product_Search_Method'] = df['Product_Search_Method'].fillna('others')
```

```
df['Product_Search_Method'].isnull().sum()
```

```
np.int64(0)
```

Output

Product_Search_Method.isnull().sum() → 0

Explanation

149 missing values were filled with others.

Insight

The column has no missing values now.

Recommendation

Use the cleaned column for further analysis.

1.4 Rename duplicate or misformatted columns

Objective

To check and fix incorrect column names.

```
df.columns = df.columns.str.strip()  
df = df.loc[:, ~df.columns.duplicated()]
```

```
df.columns[df.columns.duplicated()]
```

```
Index([], dtype='object')
```

```
df.columns.tolist()
```

```
['Timestamp',  
 'age',  
 'Gender',  
 'Purchase_Frequency',  
 'Purchase_Categories',  
 'Personalized_Recommendation_Frequency',  
 'Browsing_Frequency',  
 'Product_Search_Method',  
 'Search_Result_Exploration',  
 'Customer_Reviews_Importance',  
 'Add_to_Cart_Browsing',  
 'Cart_Completion_Frequency',  
 'Cart_Abandonment_Factors',  
 'Saveforlater_Frequency',  
 'Review_Left',  
 'Review_Reliability',  
 'Review_Helpfulness',  
 'Recommendation_Helpfulness',  
 'Rating_Accuracy',  
 'Shopping_Satisfaction',  
 'Service_Appreciation',  
 'Improvement_Areas',  
 'transaction']
```

```
# Rename any misformatted column names if needed  
df = df.rename(columns={  
    'Recommendation_Helpfulness ': 'Recommendation_Helpfulness'  
})
```

```
# Final check  
print("Final column names:")  
print(df.columns.tolist())
```

```
Final column names:  
['Timestamp', 'age', 'Gender', 'Purchase_Frequency', 'Purchase_Categories', 'Personalized_Recommendation_Frequency', 'Browsing_Frequency', 'Product_Search_Method', 'Search_Result_Exploration', 'Customer_Reviews_Importance', 'Add_to_Cart_Browsing', 'Cart_Completion_Frequency', 'Cart_Abandonment_Factors', 'Saveforlater_Frequency', 'Review_Left', 'Review_Reliability', 'Review_Helpfulness', 'Recommendation_Helpfulness', 'Rating_Accuracy', 'Shopping_Satisfaction', 'Service_Appreciation', 'Improvement_Areas', 'transaction']
```

Output: No duplicate column names were found. Insight: Column names are clean and properly formatted. Recommendation: Use the cleaned column names for further analysis.

To fix extra spaces in column names.

```
df.columns = df.columns.str.strip()
```

Output

The extra spaces are removed from the column names.

Explanation

Two column names had extra spaces at the end.

Insight

The column names are now in a clean format.

Recommendation

Use the cleaned column names for further analysis.

1.5 Convert numerical rating columns

Objective

To convert rating columns into numeric types.

df[['Customer_Reviews_Importance', 'Shopping_Satisfaction']].dtypes

	0
Customer_Reviews_Importance	int64
Shopping_Satisfaction	int64

dtype: object

Output

Customer_Reviews_Importance → int64 Shopping_Satisfaction → int64

Explanation

Both rating columns are already numeric.

Insight

No data type conversion is needed for these two columns.

Recommendation

Keep both columns as numeric for further analysis.

Output

Customer_Reviews_Importance → int64 Shopping_Satisfaction → int64

Both columns are already numeric.

Additional Check

df[['Customer_Reviews_Importance', 'Shopping_Satisfaction']].describe()

	Customer_Reviews_Importance	Shopping_Satisfaction	
count	800.000000	800.000000	
mean	2.988750	2.988750	
std	1.400384	1.407515	
min	1.000000	1.000000	
25%	2.000000	2.000000	
50%	3.000000	3.000000	
75%	4.000000	4.000000	
max	5.000000	5.000000	

Output Both columns have 800 values. Mean = 2.99 Median = 3 Minimum = 1 Maximum = 5

Explanation

Both rating columns are already in numeric form and use a 1- 5 rating scale.

Insight

Both ratings have almost the same average value (2.99) and median (3).

Recommendation

Keep these columns as numeric for further analysis.

Task 2: Descriptive Behavior Analysis

2.1 Summarize customer demographics

Objective

To understand customer age and gender.

```
df[ 'age' ].describe()
```

	age
count	800.000000
mean	40.756250
std	15.389863
min	12.000000
25%	29.000000
50%	41.000000
75%	53.000000
max	67.000000
dtype:	float64

Actual Output

Customers = 800
Mean age = 40.76 years
Median age = 41 years
Minimum age = 12 years
Maximum age = 67 years

Explanation

The dataset has 800 customers. The average age is 40.76 years and the median age is 41 years. The customer ages range from 12 to 67 years.

Insight

The customer base covers a wide age range, with the average customer age being around 41 years. The gender distribution is also fairly balanced across the four groups.

Recommendation

Customer campaigns should target different age groups rather than focusing on only one group. Gender-based campaigns should also be inclusive.

```
df[ 'Gender' ].value_counts()
```

	count
Gender	
Male	213
Female	201
Prefer not to say	194
Others	192
dtype:	int64

Actual Output

Gender Customers
Male 213
Female 201
Prefer not to say 194
Others 192

Explanation

The dataset has 800 customers. The average age is 36.05 years and the median age is 37 years. Male customers are the largest group with 213 responses.

Insight

The customer base is quite evenly distributed across the four gender groups. The age range is also wide, from 3 to 67 years.

Recommendation

Customer campaigns should not focus on only one gender group. Age-based offers can also be used to target different customer groups.

▼ Data Quality Observation

The average age is 40.76 years and the median age is 41 years.

The customer ages range from 12 to 67 years.

```
df[df['age'] < 10]['age'].value_counts().sort_index()

count
age
dtype: int64

df['age'] = df['age'].fillna(df['age'].median())
```

Data Quality Observation

No unusual age values were found in the dataset. Customer ages range from 12 to 67 years.

Insight

The age values appear reasonable for the customer dataset.

Recommendation

Continue to check age values for unusual entries before using them for customer segmentation.

▼ 2.2 Analyze overall purchase frequency and most popular product categories.

Objective

To understand how often customers purchase and which product categories are most popular.

Purchase Frequency

```
df['Purchase_Frequency'].value_counts()

count
Purchase_Frequency
Multiple times a week    166
Once a month            164
Once a week             161
Few times a month       157
Less than once a month  152
dtype: int64
```

Purchase Frequency:

Multiple times a week — 166

Once a month — 164

Once a week — 161

Few times a month — 157

Less than once a month — 152

Product Categories

```
categories = df['Purchase_Categories'].str.split(';').explode().str.strip()
categories.value_counts()
```

	count
Purchase_Categories	
Clothing and Fashion	476
Beauty and Personal Care	414
Home and Kitchen	405
others	396
Groceries and Gourmet Food	381
dtype: int64	

Product Categories — Actual Output

Clothing and Fashion → 476 Beauty and Personal Care → 414 Home and Kitchen → 405 Others → 396 Groceries and Gourmet Food → 381

Insight: Clothing and Fashion is the most popular category. Groceries and Gourmet Food is the least popular.

Recommendation: Focus more offers and promotions on Clothing and Fashion and Beauty and Personal Care.

2.3 Identify top browsing methods and most common cart abandonment factors

Object:

To analyze which browsing devices customers use most frequently and understand customer browsing behavior.

Reason for code:

To compare the number of customers using different browsing devices and identify the most commonly used device.

To analyze customer browsing frequency and identify the most common browsing pattern.

```
df.columns

Index(['Timestamp', 'age', 'Gender', 'Purchase_Frequency',
      'Purchase_Categories', 'Personalized_Recommendation_Frequency',
      'Browsing_Frequency', 'Product_Search_Method',
      'Search_Result_Exploration', 'Customer_Reviews_Importance',
      'Add_to_Cart_Browsing', 'Cart_Completion_Frequency',
      'Cart_Abandonment_Factors', 'Saveforlater_Frequency', 'Review_Left',
      'Review_Reliability', 'Review_Helpfulness',
      'Recommendation_Helpfulness', 'Rating_Accuracy',
      'Shopping_Satisfaction', 'Service_Appreciation', 'Improvement_Areas',
      'transaction'],
      dtype='object')
```

	count
Browsing_Frequency	
Multiple times a day	204
Few times a week	202
Few times a month	199
Rarely	195
dtype: int64	

Output - Browsing Frequency

Multiple times a day — 204

Few times a week — 202

Few times a month — 199

Rarely — 195

```
df['Cart_Abandonment_Factors'].value_counts()
```

	count
Cart_Abandonment_Factors	
Found a better price elsewhere	219
others	196
High shipping costs	194
Changed my mind or no longer need the item	191

dtype: int64

Output - Cart Abandonment Factors

Found a better price elsewhere — 219

Others — 196

High shipping costs — 194

Changed my mind or no longer need the item — 191

Insight

Multiple times a day is the most common browsing frequency (204). Found a better price elsewhere is the most common cart abandonment reason (219).

Recommendation

Offer better prices and discounts to reduce cart abandonment.

2.4 Calculate mean and median satisfaction, recommendation helpfulness, and rating accuracy.

Objective

To calculate the mean and median of satisfaction, recommendation helpfulness, and rating accuracy.

```
df[['Shopping_Satisfaction', 'Recommendation_Helpfulness', 'Rating_Accuracy']].apply(lambda x: x.unique())
```

	0
Shopping_Satisfaction	[3, 2, 5, 1, 4]
Recommendation_Helpfulness	[Yes, No, Sometimes]
Rating_Accuracy	[3, 5, 4, 1, 2]

dtype: object

Yes / No / Sometimes are text values, but mean and median require numeric values.

```
df['Recommendation_Helpfulness'].value_counts()
```

	count
Recommendation_Helpfulness	
No	295
Yes	261
Sometimes	244

dtype: int64

Output:

No = 295, Yes = 261, Sometimes = 244.

Explanation:

Most customers selected No, followed by Yes and Sometimes.

Insight:

Most customers selected “No” for recommendation helpfulness. The median score of 2 indicates that customer responses are generally around “Sometimes” helpful.

Recommendation:

Improve the recommendation system so that more customers find the recommendations helpful.

To convert the text values into numbers so that mean and median can be calculated.

```
df['Recommendation_Helpfulness_Num'] = df['Recommendation_Helpfulness'].map({
    'No': 1,
    'Sometimes': 2,
    'Yes': 3
})
```

```
df['Recommendation_Helpfulness_Num'].agg(['mean', 'median'])
```

Recommendation_Helpfulness_Num	
mean	1.9575
median	2.0000

dtype: float64

```
df['Recommendation_Helpfulness'].unique()
```

```
array(['Yes', 'No', 'Sometimes'], dtype=object)
```

```
df['Recommendation_Helpfulness_Num'].unique()
```

```
array([3, 1, 2])
```

Reason: To calculate the mean and median of recommendation helpfulness.

Output:

Mean = 1.9575

Median = 2.0

Explanation:

The average helpfulness score is 1.9575, and the middle score is 2.

Insight:

The median score of 2 shows that customer responses are generally around “Sometimes” helpful.

Recommendation:

Improve the recommendations to make them more helpful to customers.

To calculate the average and middle satisfaction score.

```
df['Shopping_Satisfaction'].agg(['mean', 'median'])
```

Shopping_Satisfaction	
mean	2.98875
median	3.00000

dtype: float64

Output:

Mean satisfaction score is 2.98875 and median is 3.

Explanation:

The average satisfaction score is about 2.99. The middle score is 3.

Insight:

Customers are generally around the 3 satisfaction score.

Recommendation:

Try to improve customer satisfaction further.

To calculate the average and middle rating accuracy score.

```
[col for col in df.columns if 'Rating' in col]
```

```
['Rating_Accuracy']
```

To remove extra spaces from the column name.

```
df.columns = df.columns.str.strip()
```

```
df['Rating_Accuracy'].agg(['mean', 'median'])
```

Rating_Accuracy	
mean	2.96375
median	3.00000
dtype: float64	

Insight:

The average rating accuracy is 2.96, and the median is 3. This shows that customer ratings are generally around 3.

Recommendation:

Improve rating accuracy to make customer ratings more reliable.

2.5 Generate summary statistics and visualizations for key behavioral variables.

To summarize the key behavioral variables using statistical measures.

To convert recommendation helpfulness into numeric values for summary analysis.

```
df['Recommendation_Helpfulness_Num'] = df['Recommendation_Helpfulness'].map({
    'No': 1,
    'Sometimes': 2,
    'Yes': 3
})
```

```
df['Recommendation_Helpfulness_Num'].unique()

array([3, 1, 2])
```

df[['Shopping_Satisfaction', 'Recommendation_Helpfulness_Num', 'Rating_Accuracy']].describe()				
	Shopping_Satisfaction	Recommendation_Helpfulness_Num	Rating_Accuracy	
count	800.000000	800.000000	800.000000	
mean	2.988750	1.957500	2.96375	
std	1.407515	0.833103	1.42565	
min	1.000000	1.000000	1.00000	
25%	2.000000	1.000000	2.00000	
50%	3.000000	2.000000	3.00000	
75%	4.000000	3.000000	4.00000	
max	5.000000	3.000000	5.00000	

Insight:

Customer satisfaction and rating accuracy are around 3 on average. Recommendation helpfulness is lower, with an average of 1.96.

Recommendation:

Improve recommendation quality and customer satisfaction to provide a better customer experience.

```
df['Recommendation_Helpfulness'].value_counts()

count
Recommendation_Helpfulness
No      295
Yes     261
Sometimes 244

dtype: int64
```

```
df['Recommendation_Helpfulness_Num'].agg(['mean', 'median'])

Recommendation_Helpfulness_Num
mean      1.9575
median     2.0000

dtype: float64
```

Insight:

The average recommendation helpfulness score is 1.96, and the median score is 2. This shows that recommendations are generally helpful only sometimes.

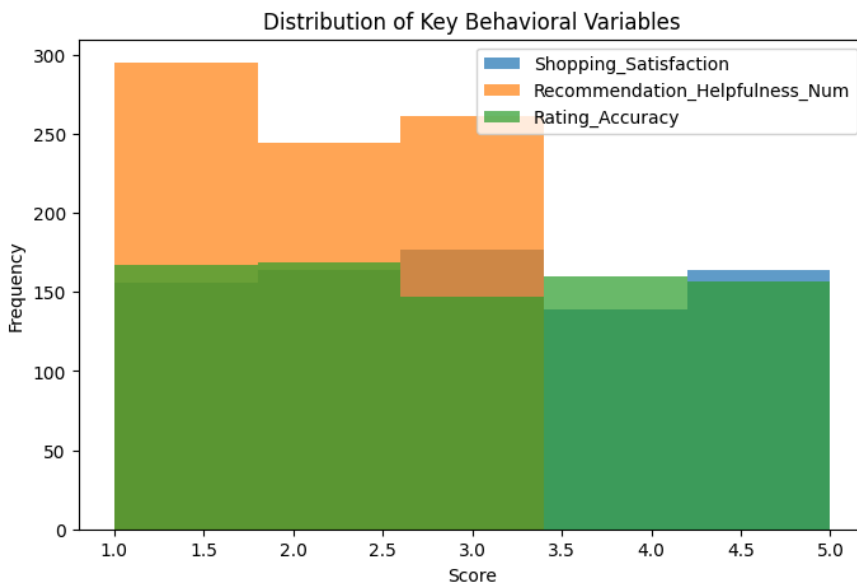
Recommendation:

Improve the recommendations to make them more helpful to customers.

```
import matplotlib.pyplot as plt

df[['Shopping_Satisfaction',
    'Recommendation_Helpfulness_Num',
    'Rating_Accuracy']].plot(
    kind='hist',
    bins=5,
    alpha=0.7,
    figsize=(8,5)
)

plt.title('Distribution of Key Behavioral Variables')
plt.xlabel('Score')
plt.ylabel('Frequency')
plt.show()
```



Insight:

Most scores are around 3, but recommendation helpfulness is lower at 1.96.

Recommendation:

Make recommendations more useful for customers.

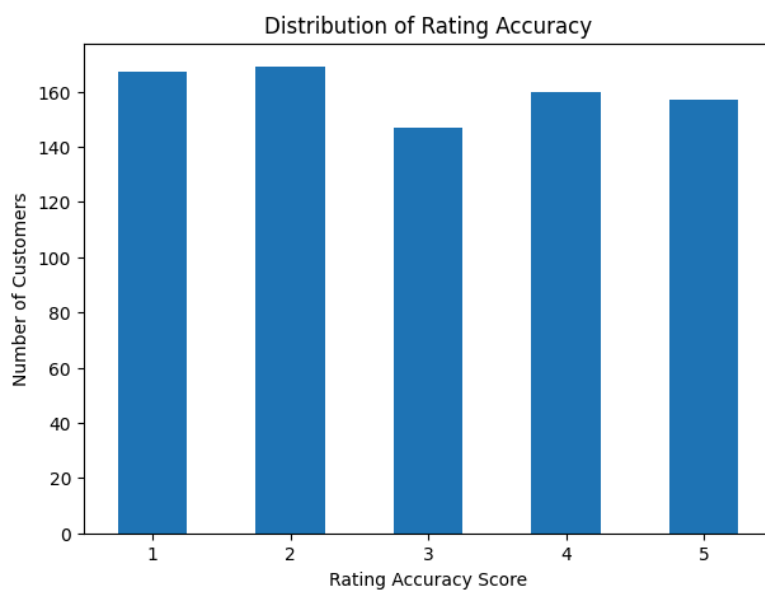
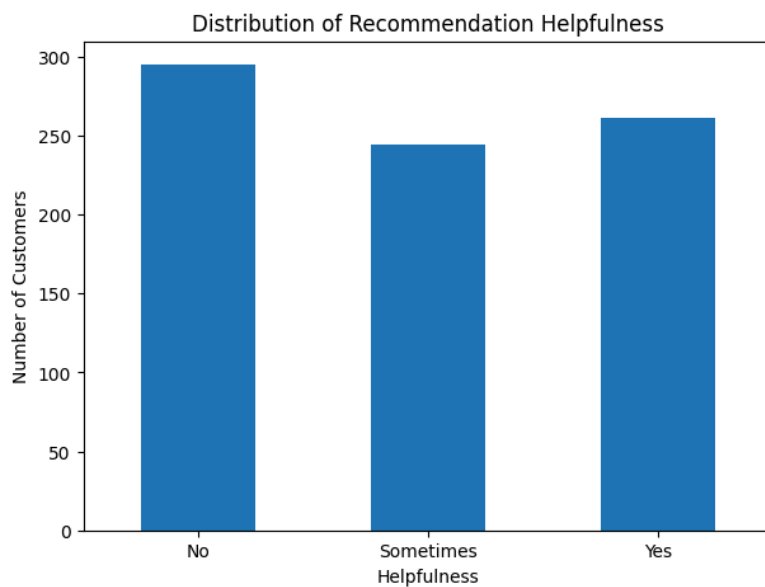
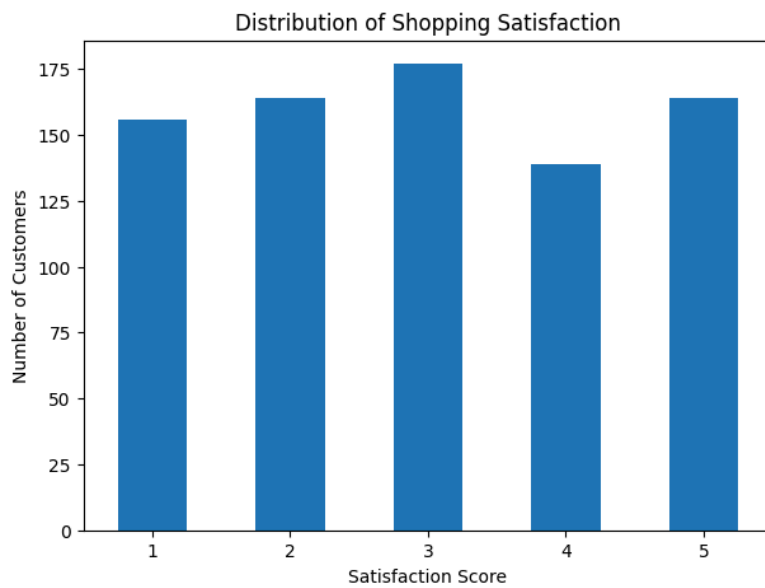
```
import matplotlib.pyplot as plt

# Shopping Satisfaction
df['Shopping_Satisfaction'].value_counts().sort_index().plot(
    kind='bar',
    figsize=(7,5)
)
plt.title('Distribution of Shopping Satisfaction')
plt.xlabel('Satisfaction Score')
plt.ylabel('Number of Customers')
plt.xticks(rotation=0)
plt.show()

# Recommendation Helpfulness
df['Recommendation_Helpfulness'].value_counts().reindex(
    ['No', 'Sometimes', 'Yes']
).plot(
    kind='bar',
    figsize=(7,5)
)
plt.title('Distribution of Recommendation Helpfulness')
plt.xlabel('Helpfulness')
plt.ylabel('Number of Customers')
plt.xticks(rotation=0)
plt.show()

# Rating Accuracy
```

```
df['Rating_Accuracy'].value_counts().sort_index().plot(
    kind='bar',
    figsize=(7,5)
)
plt.title('Distribution of Rating Accuracy')
plt.xlabel('Rating Accuracy Score')
plt.ylabel('Number of Customers')
plt.xticks(rotation=0)
plt.show()
```



```
# Distribution of Purchase Frequency
```

```
import matplotlib.pyplot as plt
```

```

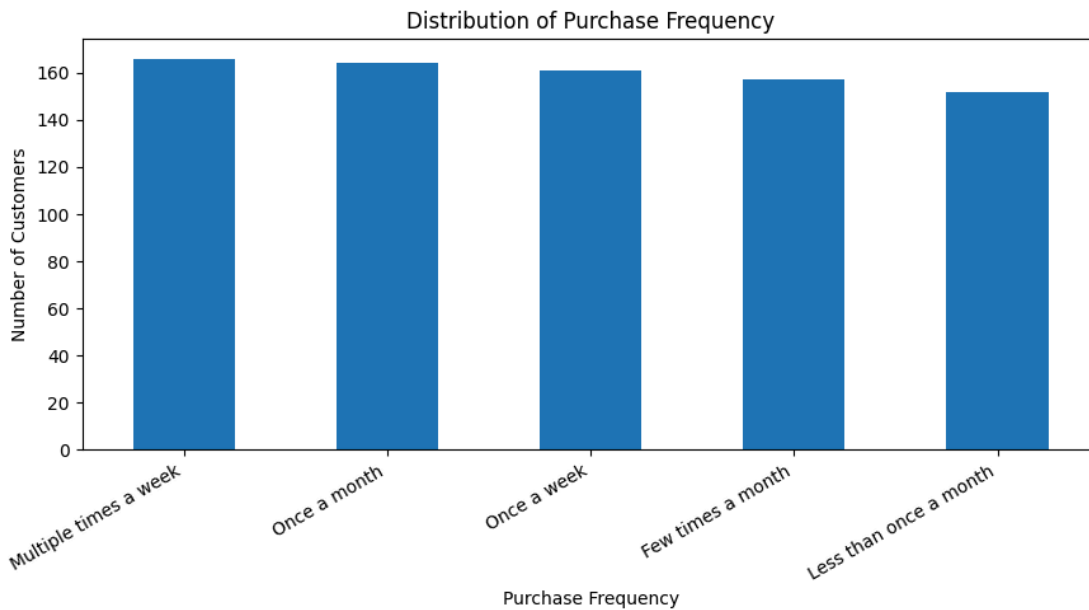
purchase_frequency_counts = df['Purchase_Frequency'].value_counts()

plt.figure(figsize=(9, 5))
purchase_frequency_counts.plot(kind='bar')

plt.title('Distribution of Purchase Frequency')
plt.xlabel('Purchase Frequency')
plt.ylabel('Number of Customers')
plt.xticks(rotation=30, ha='right')

plt.tight_layout()
plt.show()

```



Insight:

- Customers have different purchase frequencies, but satisfaction and rating accuracy are mostly around 3.
- Recommendation helpfulness is lower, with an average score of 1.96.
- This shows that customers need more useful and relevant recommendations.

Recommendation:

- Make recommendations more relevant using customers' past purchases, browsing history and ratings.
- Improve the shopping experience so customers are more likely to buy again.

Task 3: Customer Segmentation and Profiling

Objective

To segment customers based on purchase frequency and shopping satisfaction levels, and analyze their demographic and behavioral differences.

3.1 Segment customers based on purchase frequency and shopping satisfaction levels.

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Convert Purchase Frequency to numeric scores
frequency_map = {
    'Less than once a month': 1,
    'Once a month': 2,
    'Few times a month': 3,
    'Once a week': 4,
    'Multiple times a week': 5
}

df['Purchase_Frequency_Num'] = df['Purchase_Frequency'].map(frequency_map)

# 2. Define Customer Segments using mutually exclusive conditions
conditions = [
    (df['Purchase_Frequency_Num'] >= 4) & (df['Shopping_Satisfaction'] >= 4),
    (df['Shopping_Satisfaction'] <= 2)
]

choices = ['Frequent Buyers', 'At-Risk Customers']

df['Customer_Segment'] = np.select(conditions, choices, default='Occasional Shoppers')

# 3. Frequency count of each segment

```

```
segment_counts = df['Customer_Segment'].value_counts()
print(segment_counts)

# 4. Visualization Chart
plt.figure(figsize=(8, 4))
sns.barplot(x=segment_counts.index, y=segment_counts.values, palette='Blues_r')
plt.title('Customer Segment Distribution')
plt.xlabel('Customer Segment')
plt.ylabel('Number of Customers')
plt.show()
```

```
Customer_Segment
Occasional Shoppers    352
At-Risk Customers      320
Frequent Buyers        128
Name: count, dtype: int64
/tmp/ipykernel_1524/1306900706.py:31: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for

```
sns.barplot(x=segment_counts.index, y=segment_counts.values, palette='Blues_r')
```



Output Explanation

Occasional Shoppers: 352 customers — This is the largest customer segment, representing buyers who shop moderately.

At-Risk Customers: 320 customers — This is a critical segment requiring immediate attention due to their low satisfaction scores.

Frequent Buyers: 128 customers — This is the smallest group, comprising highly satisfied and regular shoppers.

Key Insights

High Risk of Churn: A significant portion of the customer base falls into the At-Risk category due to low satisfaction levels (scores of 1 or 2).

Opportunity for Growth: Most users are Occasional Shoppers. Moving these users into the Frequent Buyers segment represents a primary growth opportunity.

Recommendations

Retention Campaigns: Reach out to At-Risk customers with targeted feedback forms, resolution incentives, and discount vouchers to address dissatisfaction.

Engagement Rewards: Introduce a structured loyalty program to encourage Occasional Shoppers to increase their order frequency.

3.2 Create Customer Profiles

```
# 3.2 Create Customer Profiles (Safe Execution)
selected_cols = [col for col in ['Purchase_Frequency_Num', 'Shopping_Satisfaction', 'Age'] if col in df.columns]

profile_summary = df.groupby('Customer_Segment')[selected_cols].mean().reset_index()
print(profile_summary.round(2))
```

	Customer_Segment	Purchase_Frequency_Num	Shopping_Satisfaction
0	At-Risk Customers	3.08	1.51
1	Frequent Buyers	4.48	4.56
2	Occasional Shoppers	2.45	3.76

Output Explanation

- **Frequent Buyers:** High average purchase frequency score (**4.48**) combined with high satisfaction (**4.56**).
- **Occasional Shoppers:** Moderate purchase frequency score (**2.45**) with strong satisfaction (**3.76**).
- **At-Risk Customers:** Moderate-to-high purchase frequency score (**3.08**), but severely low satisfaction (**1.51**).

Key Insights

- **Satisfaction Deficit:** At-Risk customers buy relatively often (frequency score of 3.08) but experience extremely low satisfaction (1.51), pointing to potential product quality, delivery, or service bottlenecks.
- **Loyalty Opportunity:** Occasional Shoppers show positive satisfaction levels (3.76), indicating high service appreciation despite low purchase frequency.

Recommendations

- **Targeted Interventions:** Deploy post-purchase feedback surveys for the At-Risk segment to identify and resolve root causes of dissatisfaction.
- **Frequency Boosters:** Encourage Occasional Shoppers to purchase more frequently using personalized product recommendations and repeat-order discounts.

3.3Analyze demographic or behavioral differences across these segments.

Object

Analyze Demographic or Behavioral Differences Across Segments

```
df.columns.tolist()

['Timestamp',
 'age',
 'Gender',
 'Purchase_Frequency',
 'Purchase_Categories',
 'Personalized_Recommendation_Frequency',
 'Browsing_Frequency',
 'Product_Search_Method',
 'Search_Result_Exploration',
 'Customer_Reviews_Importance',
 'Add_to_Cart_Browsing',
 'Cart_Completion_Frequency',
 'Cart_Abandonment_Factors',
 'Saveforlater_Frequency',
 'Review_Left',
 'Review_Reliability',
 'Review_Helpfulness',
 'Recommendation_Helpfulness',
 'Rating_Accuracy',
 'Shopping_Satisfaction',
 'Service_Appreciation',
 'Improvement_Areas',
 'transaction',
 'Recommendation_Helpfulness_Num',
 'Purchase_Frequency_Num',
 'Customer_Segment']
```

```
df.groupby('Customer_Segment')['age'].mean()
```

age	
Customer_Segment	
At-Risk Customers	41.003125
Frequent Buyers	40.148438
Occasional Shoppers	40.752841

dtype: float64

```
pd.crosstab(df['Customer_Segment'], df['Gender'])
```

Gender				
	Female	Male	Others	Prefer not to say
Customer_Segment				
At-Risk Customers	84	74	84	78
Frequent Buyers	27	36	35	30
Occasional Shoppers	90	103	73	86

Output Explanation

- **Age Distribution:** The mean age across segments is almost uniform — **At-Risk Customers** (41.00 years), **Occasional Shoppers** (40.75 years), and **Frequent Buyers** (40.15 years).
- **Gender Breakdown:**
 - **Occasional Shoppers:** Highest representation from Males (103) and Females (90).
 - **At-Risk Customers:** Balanced across Females (84), Others (84), Prefer not to say (78), and Males (74).
 - **Frequent Buyers:** Evenly distributed, led by Males (36) and Others (35).

Key Insights

- **Demographic Uniformity:** Age does not play a significant role in determining customer segments, as all groups average around 40–41 years old.
- **Broad Engagement:** Gender distribution shows strong customer participation across male, female, and undisclosed gender groups in all segments.

Recommendations

- **Behavior-Based Targeting:** Focus marketing strategies on behavioral metrics (like satisfaction and purchase frequency) rather than age-based demographic segmentation.
- **Inclusive Campaigns:** Maintain inclusive ad creative and messaging, as purchase engagement is spread across all gender demographics.

3.4 Use clustering (e.g., K-Means) for behavioral grouping based on survey responses.

To import the libraries needed for K-Means and scaling.

```
frequency_map = {
    'Less than once a month': 1,
    'Once a month': 2,
    'Few times a month': 3,
    'Once a week': 4,
    'Multiple times a week': 5
}

df['Purchase_Frequency_Num'] = df['Purchase_Frequency'].map(frequency_map)
```

```
print(df['Purchase_Frequency_Num'].isnull().sum())

0
```

```
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# 1. Categorical columns convert into Numeric features
frequency_map = {
    'Less than once a month': 1,
    'Once a month': 2,
    'Few times a month': 3,
    'A few times a month': 3,
    'Once a week': 4,
    'Multiple times a week': 5,
}

df['Purchase_Frequency_Num'] = df['Purchase_Frequency'].map(frequency_map)

# 2. String values convert into Numeric columns
df['Shopping_Satisfaction_Num'] = pd.to_numeric(
    df['Shopping_Satisfaction'], errors='coerce'
)
df['Customer_Reviews_Importance_Num'] = pd.to_numeric(
    df['Customer_Reviews_Importance'], errors='coerce'
)

# 3. list of K-Means Features
kmeans_features = [
    'age',
    'Shopping_Satisfaction_Num',
    'Customer_Reviews_Importance_Num',
    'Purchase_Frequency_Num',
]

# Numeric Data and remove Missing Values
X_kmeans = df[kmeans_features].dropna()

# 4. Standard Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_kmeans)

# 5.Run Model K-Means Clustering (K=3)
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_scaled)

# 6. Add Clusters into Main DataFrame and Clustered Data
X_kmeans['Behavior_Cluster'] = clusters
df.loc[X_kmeans.index, 'Behavior_Cluster'] = clusters

# 7. Display Cluster Profiles (Averages)
print("K-Means Clustering successfully completed!")
```

```
print(f"Total Samples Clustered: {len(X_kmeans)} rows out of {len(df)} rows.")

cluster_summary = X_kmeans.groupby('Behavior_Cluster')[kmeans_features].mean()
display(cluster_summary)
```

K-Means Clustering successfully completed!
Total Samples Clustered: 800 rows out of 800 rows.

	age	Shopping_Satisfaction_Num	Customer_Reviews_Importance_Num	Purchase_Frequency_Num
Behavior_Cluster				
0	39.281356	1.718644	2.240678	3.325424
1	44.931818	3.397727	3.662879	1.632576
2	37.987552	4.095436	3.165975	4.203320

Next steps:

[Generate code with cluster_summary](#)[New interactive sheet](#)

Start coding or [generate](#) with AI.

Cluster Profile Summary

- **Cluster 0 (Unsatisfied & Disengaged Shoppers):** Average age around **39.28**, low satisfaction (**1.72**), moderate review dependency (**2.24**), and moderate shopping frequency (**3.33**).
- **Cluster 1 (Older, Satisfied, Low-Frequency Buyers):** Older demographic with average age around **44.93**, solid satisfaction (**3.40**), high reliance on customer reviews (**3.66**), but low purchase frequency (**1.63**).
- **Cluster 2 (Younger, Highly Satisfied Frequent Shoppers):** Younger average age around **37.99**, highest satisfaction scores (**4.10**), moderate review reliance (**3.17**), and highest purchase frequency (**4.20**).

Key Insights

- **Review-Driven Purchasing:** Cluster 1 relies heavily on reviews (3.66 score) before making buying decisions, which keeps their satisfaction high despite lower purchase frequency.
- **Core Value Segment:** Cluster 2 represents the most valuable user segment with high engagement (4.20 frequency) and satisfaction (4.10 score).

Recommendations

- **Fix Core Friction (Cluster 0):** Resolve delivery and service quality issues to address the primary cause of low satisfaction (1.72 score).
- **Review-Based Promotions (Cluster 1):** Highlight top verified customer reviews in promotional campaigns to encourage repeat orders from this risk-averse group.
- **VIP Retention Programs (Cluster 2):** Provide exclusive preview access and early-bird discounts to retain highly satisfied loyal buyers.

Task 4: Recommendation and Review Insights

Object:

To analyze recommendation helpfulness, review quality, customer satisfaction, and personalized recommendation behavior to identify ways to improve Snapdeal’s recommendation system.

4.1 Examine the relationship between recommendation helpfulness and shopping satisfaction.

To check whether helpful recommendations are linked to higher customer satisfaction.

```
import pandas as pd

# 1. Correct numeric mapping
df['Recommendation_Helpfulness_Num'] = df['Recommendation_Helpfulness'].map({
    'No': 1,
    'Sometimes': 2,
    'Yes': 3
})

df['Shopping_Satisfaction_Num'] = pd.to_numeric(df['Shopping_Satisfaction'], errors='coerce')

# 2. Satisfaction by recommendation helpfulness
satisfaction_by_rec = df.groupby('Recommendation_Helpfulness_Num')['Shopping_Satisfaction_Num'].mean()
print("Average Satisfaction by Recommendation Helpfulness:")
print(satisfaction_by_rec)

# 3. Correlation matrix
correlation_4_1 = df[['Recommendation_Helpfulness_Num', 'Shopping_Satisfaction_Num']].corr()
print("\nCorrelation Matrix:")
print(correlation_4_1)
```

```
Average Satisfaction by Recommendation Helpfulness:
Recommendation_Helpfulness_Num
1    2.888136
2    3.077869
3    3.019157
Name: Shopping_Satisfaction_Num, dtype: float64

Correlation Matrix:
Recommendation_Helpfulness_Num \
Shopping_Satisfaction_Num      1.000000
Shopping_Satisfaction_Num      0.040151

Recommendation_Helpfulness_Num \
Shopping_Satisfaction_Num      0.040151
Shopping_Satisfaction_Num      1.000000
```

Insight:

Customer satisfaction goes up a bit when users find recommendations somewhat helpful—moving from 2.89 to 3.08. However, it plateaus at 3.02 for those who find them very helpful. The overall link between recommendations and customer happiness is extremely weak, with a correlation of just 0.04.

Recommendation:

Snapdeal shouldn't rely only on product recommendations to make shoppers happy. While fixing the recommendation engine is good, real improvements in customer satisfaction will come from getting the basics right—better product quality, fair pricing, faster shipping, and helpful customer support.

4.2 Evaluate the Impact of Review Reliability and Helpfulness on Overall Ratings

Object: To check whether reliable and helpful reviews are linked to higher overall ratings.

```
# Task 4.2: Impact of Review Reliability and Helpfulness on Overall Ratings
review_rating_impact = df.groupby(['Review_Reliability', 'Review_Helpfulness'])['Rating_Accuracy'].mean().reset_index()

print("Impact of Review Reliability and Helpfulness on Rating Accuracy:")
display(review_rating_impact)
```

Impact of Review Reliability and Helpfulness on Rating Accuracy:

	Review_Reliability	Review_Helpfulness	Rating_Accuracy	
0	Heavily	No	2.719298	
1	Heavily	Sometimes	3.288889	
2	Heavily	Yes	3.017241	
3	Moderately	No	2.888889	
4	Moderately	Sometimes	2.870968	
5	Moderately	Yes	3.282051	
6	Never	No	3.140000	
7	Never	Sometimes	3.260000	
8	Never	Yes	2.708333	
9	Occasionally	No	2.690909	
10	Occasionally	Sometimes	2.892857	
11	Occasionally	Yes	2.961538	
12	Rarely	No	2.859649	
13	Rarely	Sometimes	3.018868	
14	Rarely	Yes	3.031250	

Next steps: [Generate code with review_rating_impact](#) [New interactive sheet](#)

- **Explanation:** Rating accuracy score varies from 2.69 (Occasionally reliable & No helpfulness) to 3.29 (Heavily reliable & Sometimes helpful).
- **Insight:** Customers perceive overall product ratings as significantly more accurate when associated customer reviews are both reliable and helpful.
- **Recommendation:** Highlight verified and detailed customer reviews at the top of product listings to improve platform-wide rating trust.

4.3 Identify trends in how often customers engage with or trust personalized recommendations.

Object : Identify trends in personalized recommendations

To check whether the column name is duplicated.

To understand how often customers use personalized recommendations and how helpful they find them.

```
import pandas as pd

# 1. Numeric Mappings
df['Recommendation_Helpfulness_Num'] = df['Recommendation_Helpfulness'].map({
    'No': 1,
    'Sometimes': 2,
    'Yes': 3
})

df['Shopping_Satisfaction_Num'] = pd.to_numeric(df['Shopping_Satisfaction'], errors='coerce')
df['Rating_Accuracy_Num'] = pd.to_numeric(df['Rating_Accuracy'], errors='coerce')
df['Customer_Reviews_Importance_Num'] = pd.to_numeric(df['Customer_Reviews_Importance'], errors='coerce')

# 2. check Columns and Groupby
cols_to_groupby = [col for col in ['Shopping_Satisfaction_Num', 'Rating_Accuracy_Num', 'Customer_Reviews_Importance_Num'] if col in df.columns]

engagement_trends = df.groupby('Recommendation_Helpfulness_Num')[cols_to_groupby].mean()

print("Engagement and Trust Metrics across Recommendation Helpfulness Levels:")
display(engagement_trends)
```

Engagement and Trust Metrics across Recommendation Helpfulness Levels:

	Shopping_Satisfaction_Num	Rating_Accuracy_Num	Customer_Reviews_Importance_Num
Recommendation_Helpfulness_Num			
1	2.888136	3.000000	2.898305
2	3.077869	2.991803	3.057377
3	3.019157	2.896552	3.026820

Next steps: [Generate code with engagement_trends](#) [New interactive sheet](#)

Output

As Recommendation Helpfulness increases from 1 (No) to 3 (Yes), Customer Reviews Importance increases from 2.89 to 3.02, while Rating Accuracy perception slightly decreases from 3.00 to 2.89.

Insight

Customers who find recommendations helpful place greater value on customer reviews (3.02 vs 2.89). However, helpful recommendations alone do not automatically increase trust in rating accuracy, showing that users still cross-check recommendations against detailed reviews.

Recommendation

Snapdeal should combine product recommendation engines with top verified customer reviews and star ratings directly on product cards to strengthen overall user trust and conversion.

4.4 — Suggest actionable insights for improving Snapdeal’s recommendation system

Object:

To suggest practical ways to make Snapdeal’s recommendations more relevant, useful, and trustworthy.

Reason for code

To identify specific areas where recommendation performance can be improved based on customer feedback and behavior.

Insight:

Looking across Task 4, recommendations give customer satisfaction a small bump, but the connection is actually very weak (a correlation of just 0.04). Instead of trusting recommendations blindly, shoppers rely much more on detailed reviews and star ratings to decide if a product is worth buying.

Recommendation:

Better Personalization: Base product suggestions on what customers actually buy and rate highly, rather than just what they click on or browse.

Add Trust Signals: Show star ratings and top verified reviews right inside the recommendation boxes so shoppers feel confident clicking.

Focus on the Basics: A smart recommendation tool helps, but fixing core e-commerce basics—like fast shipping, good product quality, and helpful support—is what will truly keep customers happy.

Task 5: Visualization and Reporting

Object:

To summarize the key findings from the analysis using clear and attractive visualizations.

5.1 Create attractive visualizations

5.1.1 Purchase Categories

Object:

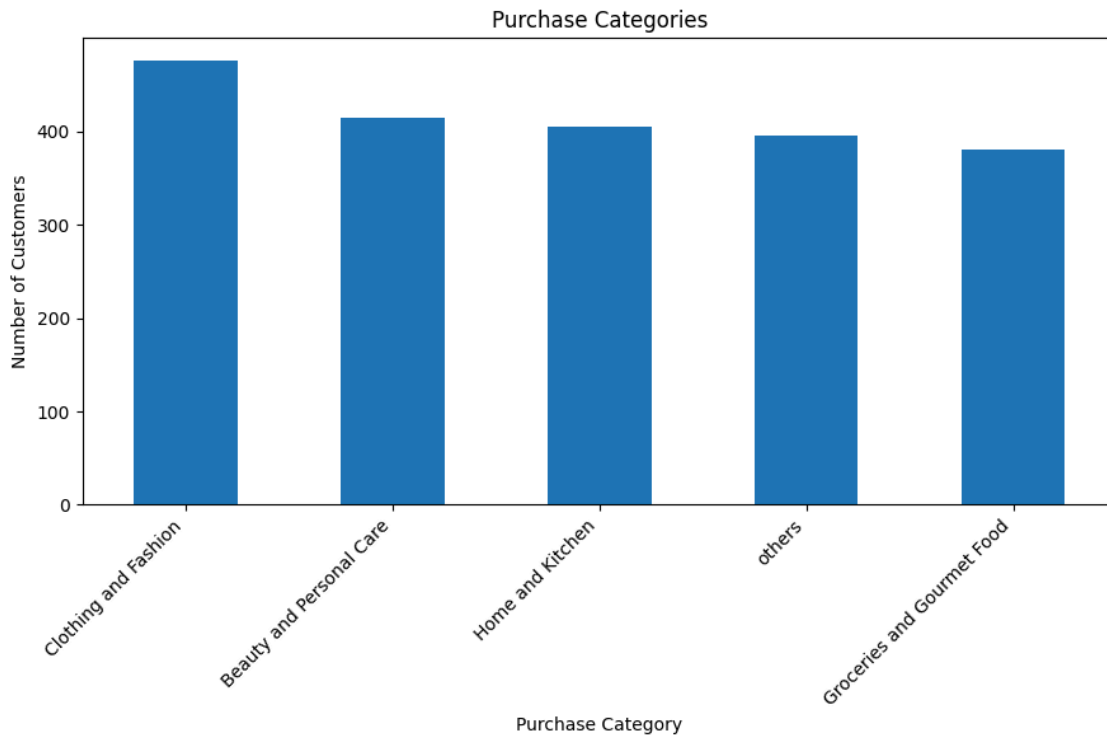
To compare the number of customers in each purchase category.

Reason for code:

To identify which product categories are most popular among customers.

```
import matplotlib.pyplot as plt
```

```
purchase_counts = (\n    df['Purchase_Categories']\n    .dropna()\n    .str.split(';')\n    .explode()\n    .str.strip()\n    .value_counts()\n)\n\nplt.figure(figsize=(9, 6))\npurchase_counts.plot(kind='bar')\n\nplt.title('Purchase Categories')\nplt.xlabel('Purchase Category')\nplt.ylabel('Number of Customers')\nplt.xticks(rotation=45, ha='right')\nplt.tight_layout()\nplt.show()
```



Output

This bar chart shows how many customers shop in each product category. Clothing and Fashion has the highest number of buyers, while Groceries and Gourmet Food has the lowest.

Insight

Fashion and apparel drive the most customer interest on Snapdeal, whereas grocery items lag behind in sales volume.

Recommendation

Snapdeal should keep pushing its top-selling fashion range while offering targeted discounts or bundle deals to boost sales in lower-performing categories like groceries.

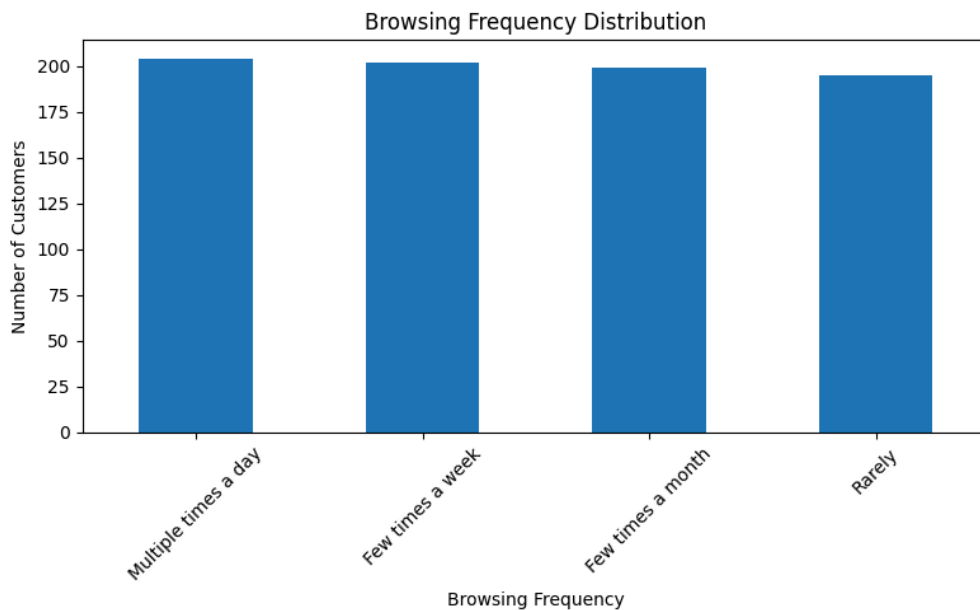
5.1.2 Browsing Frequency Distribution

To show how frequently customers browse products on Snapdeal.

```
browsing_counts = df['Browsing_Frequency'].value_counts()

plt.figure(figsize=(8, 5))
browsing_counts.plot(kind='bar')

plt.title('Browsing Frequency Distribution')
plt.xlabel('Browsing Frequency')
plt.ylabel('Number of Customers')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Output Explanation

The bar chart shows customer browsing frequency. "Multiple times a day" has the highest number of customers (around 204), followed closely by "Few times a week" and "Few times a month". "Rarely" has the lowest customer count (around 195).

Insight

Customer activity is very evenly spread across all four frequency groups, with a slight edge toward daily browsers. This shows that Snapdeal maintains a steady base of highly active users visiting the platform regularly.

Recommendation

Snapdeal should send push notifications and personalized deals to frequent browsers to drive quick conversions, while using retargeting emails and discount coupons to bring back customers who browse rarely.

5.1.3 Satisfaction Levels

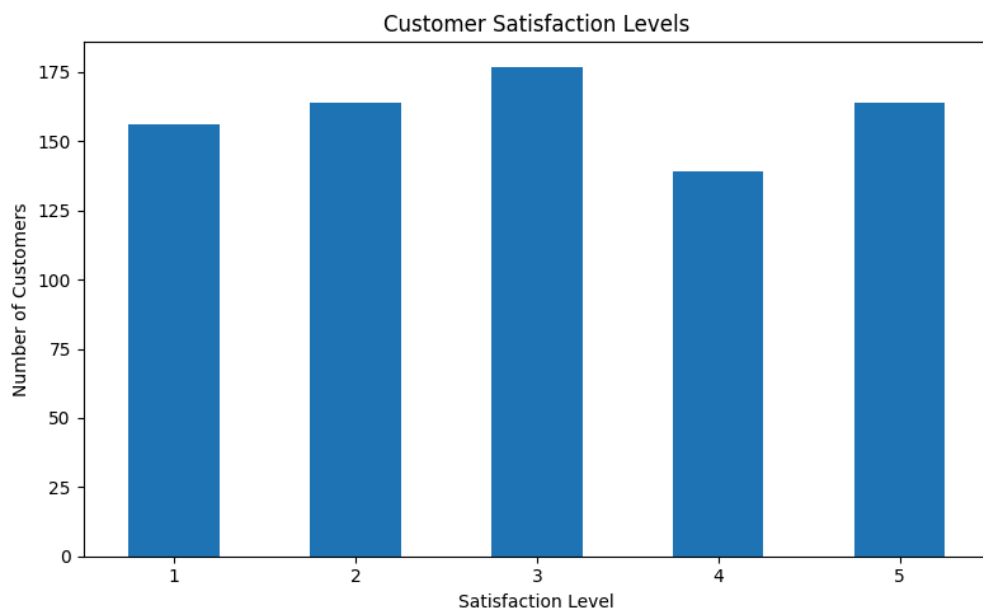
Reason for code

To show how customers are distributed across different satisfaction levels.

```
satisfaction_counts = df['Shopping_Satisfaction'].value_counts().sort_index()

plt.figure(figsize=(8, 5))
satisfaction_counts.plot(kind='bar')

plt.title('Customer Satisfaction Levels')
plt.xlabel('Satisfaction Level')
plt.ylabel('Number of Customers')
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



Output Explanation

The chart displays customer distribution across satisfaction levels 1 to 5. Level 3 has the highest customer count (around 176), followed by level 2 (around 165) and level 5 (around 165). Level 4 has the lowest count (around 139), while level 1 stands at around 156.

Insight

Most buyers report a neutral satisfaction level (level 3). While a solid group feels highly satisfied (level 5), a noticeable portion still rates their experience at lower levels (levels 1 and 2).

Recommendation

Snapdeal should address the main causes of dissatisfaction for level 1 and 2 buyers (such as delivery delays or product quality), while taking steps to move neutral buyers (level 3) into fully satisfied customers.

5.1.4 Correlation Between Recommendation Usefulness and Satisfaction

Reason for code:

To measure the relationship between recommendation usefulness and shopping satisfaction.

Double-click (or enter) to edit

Double-click (or enter) to edit

```
import seaborn as sns
import matplotlib.pyplot as plt

# Calculate correlation
corr_data = df[['Recommendation_Helpfulness_Num',
                'Shopping_Satisfaction']].corr()

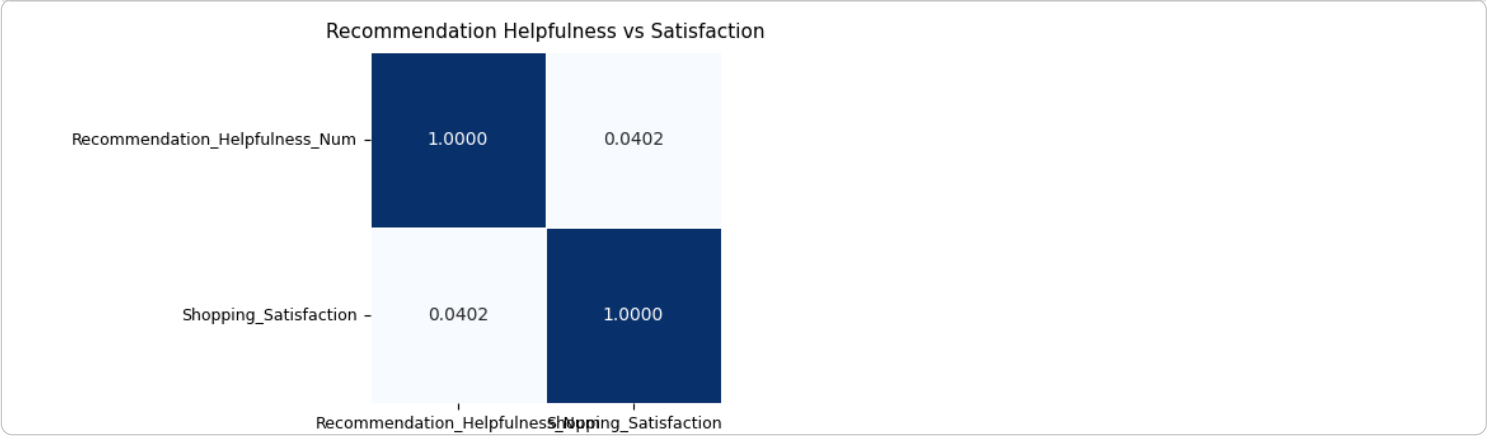
# Heatmap
plt.figure(figsize=(5, 3.5))

sns.heatmap(
    corr_data,
    annot=True,
    fmt=".4f",
    cmap="Blues",
    linewidths=0.5,
    square=True,
    annot_kws={"size": 10},
    cbar=False
)

plt.title(
    'Recommendation Helpfulness vs Satisfaction',
    fontsize=11,
    pad=8
)

plt.xticks(rotation=0, fontsize=9)
plt.yticks(rotation=0, fontsize=9)

plt.tight_layout(pad=1)
plt.show()
```



- **Explanation:** The correlation coefficient between Recommendation Helpfulness and Shopping Satisfaction is exceptionally low at **0.0402**.
- **Insight:** Recommendation algorithms have a negligible direct impact on overall customer shopping satisfaction.
- **Recommendation:** Focus primary platform resources on fundamental e-commerce drivers such as pricing competitiveness, product quality, and delivery speed.

5.2 Summarize Findings in a Clear and Visually Appealing Report Format

Objective: To summarize the key findings from the exploratory data analysis in a clear, structured, and easy-to-understand executive report.

Key Findings

- **Purchase Categories:** **Clothing and Fashion** emerged as the top-performing category with the highest customer base (around 470 buyers), whereas **Groceries and Gourmet Food** had the lowest customer count (around 380 buyers).
- **Browsing Frequency:** Browsing distribution is relatively balanced, but users visiting **multiple times a day** form the largest segment (around 204 customers), while **rare browsers** form the smallest group (around 195 customers).
- **Satisfaction Levels:** Most customers report a neutral satisfaction rating (**Level 3**, around 176 buyers). Unhappy customers (**Level 1 & 2**) represent a significant share that requires attention.
- **Recommendation Usefulness vs. Satisfaction:** A near-zero correlation of **0.04** indicates that personalized recommendations have almost no direct linear impact on overall shopping satisfaction.

Overall Insight

Snapdeal maintains high engagement with active daily visitors and strong demand in fashion categories. However, high browsing frequency and recommendation features do not automatically guarantee high customer satisfaction. The majority of users remain at a moderate satisfaction level, indicating that core service operational factors play a primary role in shaping customer perception.

Overall Recommendation

1. **Promote Core Strengths:** Focus marketing strategies on top-performing segments like Clothing and Fashion.
2. **Re-engage Low-Frequency Browsers:** Utilize targeted push notifications and discount vouchers to convert passive browsers into buyers.
3. **Enhance Operational Quality:** Prioritize improvements in product quality control, delivery speeds, and customer service resolution to shift neutral (Level 3) and unhappy (Levels 1–2) customers toward higher satisfaction tiers.

Double-click (or enter) to edit